

Laboratory Work 1

Part 1. Key Identification

Task 1.1. Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

Superkeys (at least 6)

- {EmpID}
- {SSN}
- {Email}
- {EmpID, Name}
- {EmpID, SSN}
- {SSN, Department}
- {EmpID, SSN, Email, Phone, Name, Department, Salary} (the whole row)

Candidate keys

{EmpID}, {SSN}, {Email} — each one alone uniquely identifies an employee and is minimal

Primary key

EmpID. SSN is sensitive personal data and should not be used as a key. Email can change over time. EmpID is stable, non-sensitive, and fully controlled by the company.

Can two employees share the same phone number?

Yes. In the sample data the phone values happen to be unique, but nothing enforces uniqueness in general: employees could share a department landline. Phone is not a candidate key.

Relation B: Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

Minimum primary key

{StudentID, CourseCode, Section, Semester, Year}

Why each attribute is needed

- StudentID — distinguishes students
- CourseCode — distinguishes courses
- Section — a course can have several parallel sections in one semester
- Semester, Year — a student may retake the same course in a different semester/year

Additional candidate keys

None — no other attribute combination guarantees uniqueness.

Task 1.2. Foreign Keys (university system)

Table.Attribute (FK)	References
Student.AdvisorID	Professor.ProfID
Course.DepartmentCode	Department.DeptCode

Department.ChairID	Professor.ProfID
Enrollment.StudentID	Student.StudentID
Enrollment.CourseID	Course.CourseID

Part 2. ER Diagrams

Task 2.1. Hospital Management System

Entities

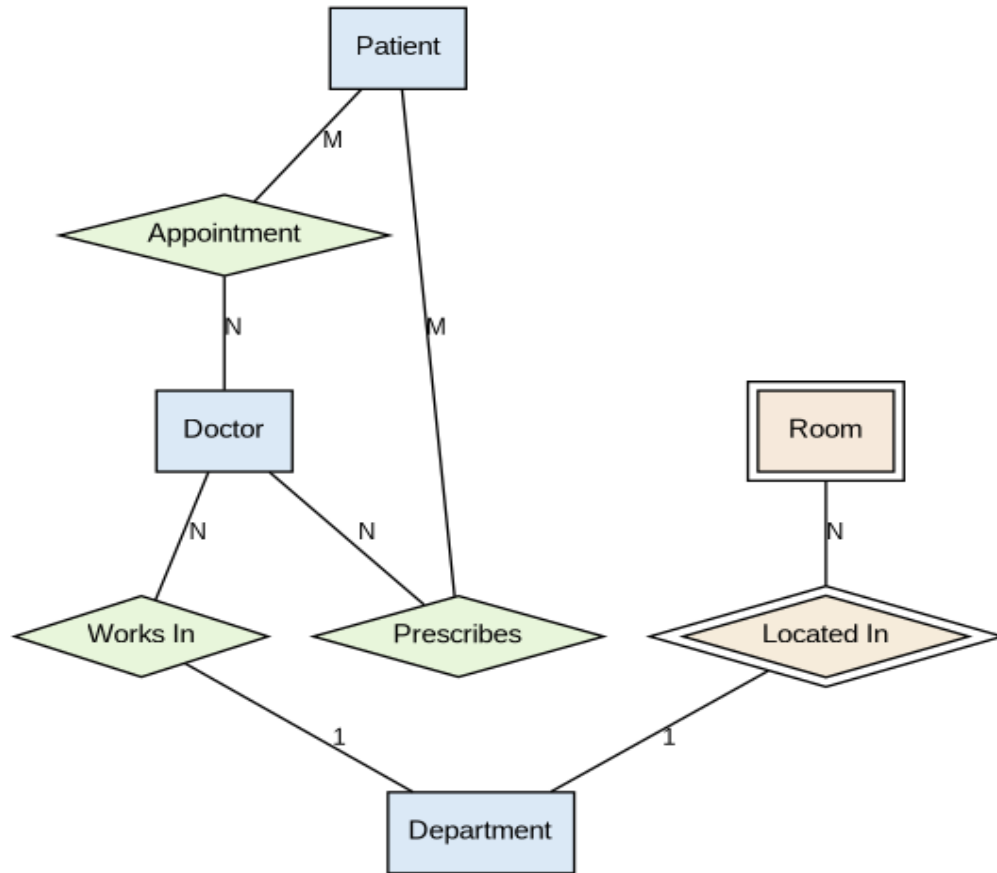
Entity	Type	Primary key
Patient	strong	PatientID
Doctor	strong	DoctorID
Department	strong	DeptCode
Room	weak	DeptCode + RoomNumber (room 101 in different departments = different rooms)

Attributes

- Patient: PatientID (PK), Name (simple), Birthdate (simple), Address (composite: Street, City, State, Zip), PhoneNumbers (multi-valued), InsuranceInfo (composite)
- Doctor: DoctorID (PK), Name, Specializations (multi-valued), Phone, OfficeLocation
- Department: DeptCode (PK), DeptName, Location
- Room: RoomNumber (partial key), Department (owner) — composite key
- Appointment (relationship attributes): AppointmentID, DateTime, Purpose, Notes
- Prescription (relationship attributes): PrescriptionID, Medication, Dosage, Instructions

Relationships and cardinality

- Department 1 : N Doctor (Works In)
- Department 1 : N Room (Located In) — identifying relationship
- Patient M : N Doctor through Appointment
- Doctor M : N Patient through Prescribes (Prescription)



Task 2.2. E-commerce Platform

Weak entity

OrderItem — has no existence without a parent Order and no independent identifier of its own; its key is composite: (OrderID, ProductID).

Many-to-many relationship with attributes

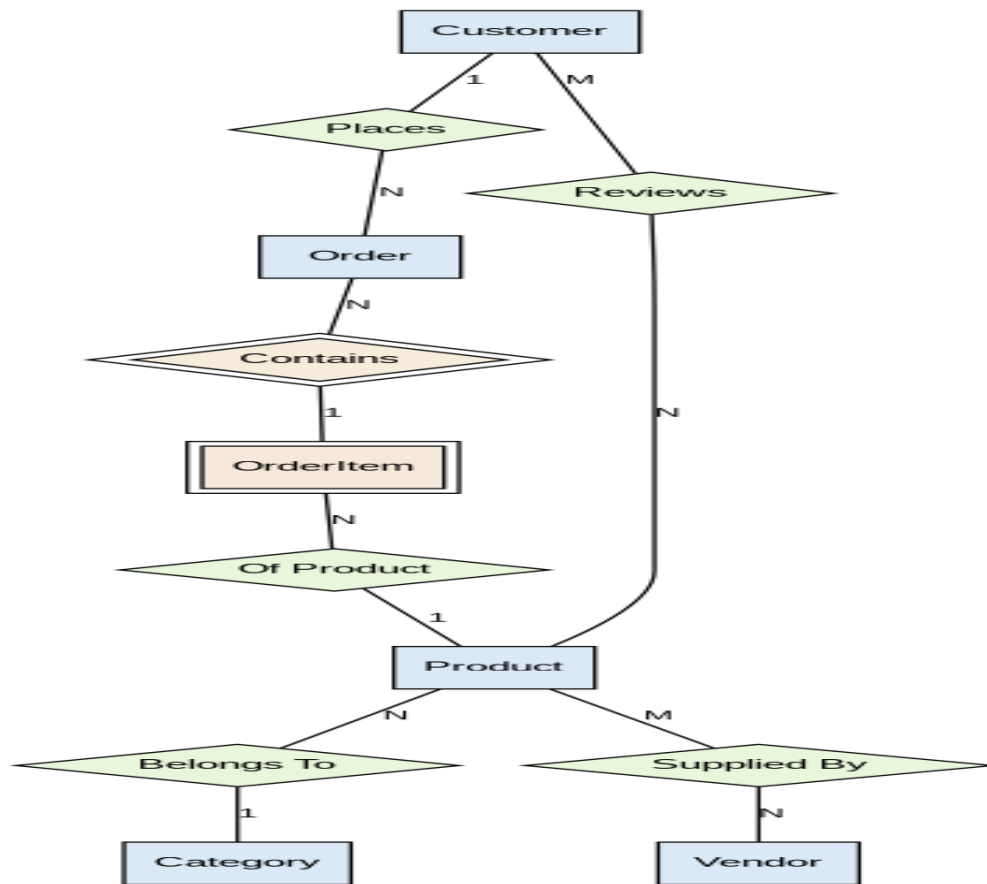
Order — Product through OrderItem: attributes Quantity and PriceAtOrder (may differ from the product's current price).

Relationships and cardinality

- Customer 1 : N Order (Places)
- Order 1 : N OrderItem (Contains) — identifying
- OrderItem N : 1 Product
- Product N : 1 Category (Belongs To)
- Product M : N Vendor (Supplied By)
- Customer M : N Product through Reviews (attributes: Rating, Comment)

Inventory (stock level) is modeled as a StockQty attribute of Product; a separate entity is not needed.

ShippingAddress is stored on Order, separate from the customer's BillingAddress.



Part 4. Normalization

Task 4.1. StudentProject

Functional dependencies

- $\text{StudentID} \rightarrow \text{StudentName}, \text{StudentMajor}$
- $\text{ProjectID} \rightarrow \text{ProjectTitle}, \text{ProjectType}, \text{SupervisorID}$
- $\text{SupervisorID} \rightarrow \text{SupervisorName}, \text{SupervisorDept}$
- $(\text{StudentID}, \text{ProjectID}) \rightarrow \text{Role}, \text{HoursWorked}, \text{StartDate}, \text{EndDate}$

Problems

Redundancy: student name/major repeat for every project the student is on; project title/type/supervisor repeat for every participant; supervisor name/department repeat for every project they supervise.

- Update anomaly: changing a student's major requires updating every one of their rows — easy to miss one and create inconsistency.
- Insert anomaly: a new project cannot be added until at least one student is assigned to it.
- Delete anomaly: removing the only participant of a project loses all information about that project (title, type, supervisor).

1NF

No violations — all attributes are atomic, no repeating groups.

2NF

Primary key: (StudentID, ProjectID). Partial dependencies: StudentID $\rightarrow$ StudentName, StudentMajor; ProjectID $\rightarrow$ ProjectTitle, ProjectType, SupervisorID.

2NF decomposition:

Student(StudentID, StudentName, StudentMajor)

Project(ProjectID, ProjectTitle, ProjectType, SupervisorID)

Assignment(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

3NF

Transitive dependency: ProjectID $\rightarrow$ SupervisorID $\rightarrow$ SupervisorName, SupervisorDept (supervisor name/department depend on SupervisorID, not directly on ProjectID).

Final 3NF decomposition:

Student(StudentID, StudentName, StudentMajor)

Supervisor(SupervisorID, SupervisorName, SupervisorDept)

Project(ProjectID, ProjectTitle, ProjectType, SupervisorID*)

Assignment(StudentID*, ProjectID*, Role, HoursWorked, StartDate, EndDate)

* — foreign key.

Task 4.2. CourseSchedule

Under the given business rules, CourseID is treated as one specific course section (one instructor, one time, one room).

Primary key

(StudentID, CourseID) — the table records the fact 'a student is enrolled in a section'; a student can take many sections, a section can have many students.

Functional dependencies

- StudentID $\rightarrow$ StudentMajor
- CourseID $\rightarrow$ CourseName, InstructorID, TimeSlot, Room
- InstructorID $\rightarrow$ InstructorName
- Room $\rightarrow$ Building (rooms are unique across the whole campus)

BCNF check

Not in BCNF: the determinants StudentID, CourseID, InstructorID, and Room are not superkeys (the only superkey of the table is {StudentID, CourseID}).

Step-by-step decomposition to BCNF

1) Split on StudentID $\rightarrow$ StudentMajor:

Student(StudentID, StudentMajor)

R1(StudentID, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)

2) Split R1 on CourseID $\rightarrow$ CourseName, InstructorID, TimeSlot, Room (and transitively InstructorName, Building):

Enrollment(StudentID, CourseID)

CourseSection(CourseID, CourseName, InstructorID, TimeSlot, Room)

3) CourseSection still has InstructorID $\rightarrow$ InstructorName (not a superkey) — split off:

Instructor(InstructorID, InstructorName)

CourseSection(CourseID, CourseName, InstructorID, TimeSlot, Room)

4) Room → Building remains (not a superkey) — split off:

RoomInfo(Room, Building)

CourseSection(CourseID, CourseName, InstructorID, TimeSlot, Room)

Final BCNF tables

Student(StudentID, StudentMajor)

Enrollment(StudentID, CourseID)

CourseSection(CourseID, CourseName, InstructorID, TimeSlot, Room)

Instructor(InstructorID, InstructorName)

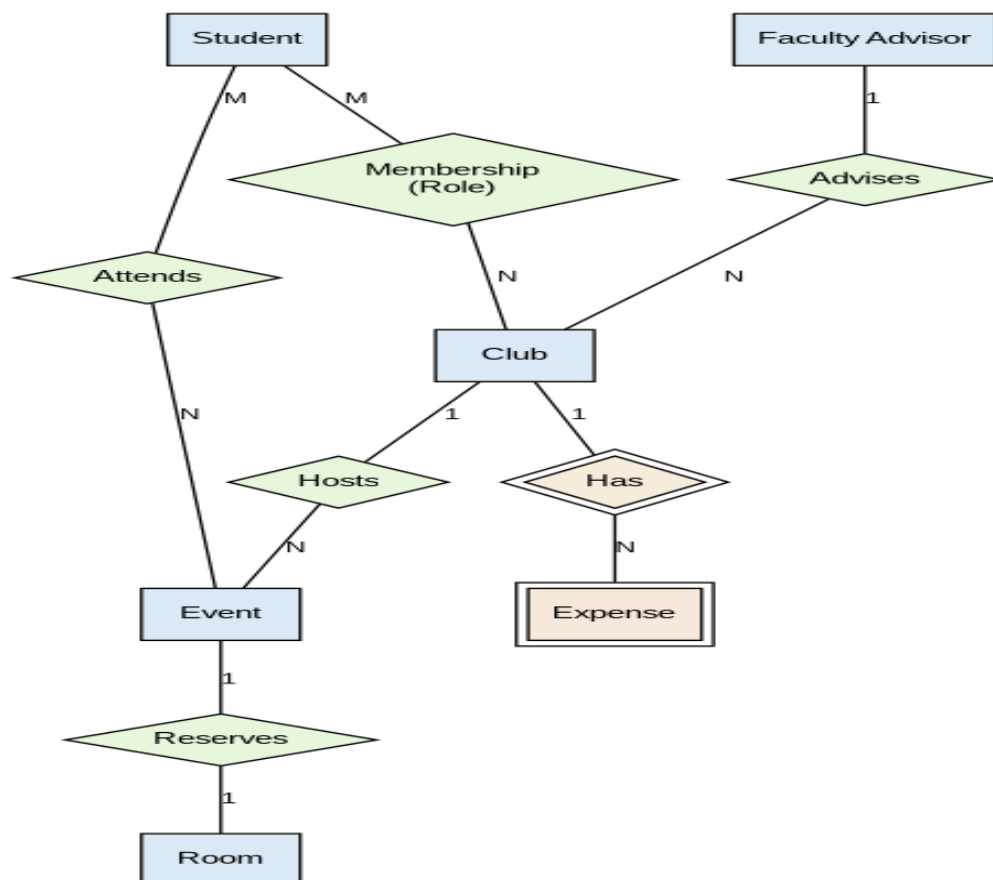
RoomInfo(Room, Building)

Loss of information

The decomposition is lossless: at every step the determinant attribute (CourseID, InstructorID, Room) stays common to both new tables, so the original data can be reconstructed with a natural join. All functional dependencies are preserved and can be checked within the individual tables.

Part 5. Design Challenge — Student Clubs

ER Diagram



Normalized relational schema

Student(StudentID, Name, Email)

FacultyAdvisor(AdvisorID, Name, Department)

Club(ClubID, Name, Budget, AdvisorID*)

Membership(StudentID*, ClubID*, JoinDate, Role)

Event(EventID, Title, DateTime, ClubID*)

Attendance(StudentID*, EventID*, Status)

Room(RoomID, Building, Capacity)

RoomReservation(ReservationID, EventID*, RoomID*, DateTime)

Expense(ExpenseID, ClubID*, Amount, Date, Description)

* — foreign key. Role in Membership stores the officer position (Member / President / Treasurer / Secretary).

Design decision

Officer positions could be modeled two ways: (a) a Role attribute directly on Membership, or (b) a separate Officer entity with term start/end dates. Option (a) was chosen — a simpler schema with fewer tables; the requirements do not ask for a history of position changes, so the extra complexity is not justified.

Example queries

- Find all students who are officers in the Computer Science Club.
- List all events scheduled for next week with their room reservations.
- Compute the total expenses of each club for the current year.